COCO: Cognitive Operating System with Continuous Oversight for Multi-Agent Workflow Reliability

Churong Liang, Jinling Gan, Kairan Hong, Qiushi Tian, Zongze Wu, and Runnan Li*

Beijing University of Posts and Telecommunications

Abstract. Multi-agent AI systems decompose complex tasks across specialized models, but this modularity introduces structural vulnerability: upstream errors cascade and amplify through downstream agents, producing subtly incorrect outputs that undermine human operators' trust and decision quality. We introduce **COCO** (**C**ognitive **O**perating System with **C**ontinuous **O**versight), a system-level reliability framework that operates transparently beneath the user interface, shielding human operators from propagated errors. COCO adopts a decoupled asynchronous architecture that isolates error detection from the critical execution path, maintaining $O(1)$ overhead. Three mechanisms address distinct failure modes: (1) Contextual Rollback Mechanism (CRM) performs history-aware state recovery to halt stochastic cascading; (2) Bidirectional Reflection Protocol (BRP) achieves stable correction consensus with bounded cost; and (3) Heterogeneous Cross-Validation (HCV) leverages cross-model disagreement to detect systematic biases. Integration tests in customer service and postal service workflow simulations demonstrate COCO's practical reliability gains. Extensive experiments show a 5.7% average improvement, with COCO achieving 95.1% of large-model performance at 30× parameter reduction, establishing it as a scalable technical substrate for reliable multi-agent pipelines in human-facing applications.

Keywords: Human-AI teaming · Multi-agent systems · Error correction · Trust calibration · Asynchronous monitoring.

1 Introduction

In a typical customer service center, a human agent fields dozens of complex inquiries daily—refund disputes, policy clarifications, delivery exceptions. Increasingly, these agents rely on multi-agent AI systems that decompose each inquiry into subtasks: intent classification, policy retrieval, historical case matching, and response drafting—each handled by a specialized model. However, when the upstream intent classifier misinterprets "refund eligibility" as "return policy," the

* Corresponding author.

downstream retrieval agent fetches irrelevant documents, and the response agent generates a fluent but substantively incorrect reply. The human agent is left with an output that appears coherent yet is fundamentally wrong—precisely the condition HCI research identifies as most harmful to trust calibration: errors that are difficult to detect lead to dangerous overreliance or costly disuse [2, 3, 15]. This scenario is not hypothetical; it reflects operational challenges we observed in real customer service and postal service deployments.

The HCI Gap in Multi-Agent Reliability. HCI research has extensively documented how single-model AI errors shape user trust, decision quality, and willingness to rely on automation [15, 23, 1]. A central finding is that error subtlety—not just error frequency—determines trust outcomes: when users cannot distinguish correct from incorrect outputs, neither vigilance nor trust is appropriately calibrated. However, this literature has focused almost exclusively on single-model interactions. Multi-agent architectures introduce a structurally distinct challenge: modular decomposition creates pathways for error amplification that are invisible at the interface level. The human agent sees only the final output, not the upstream misclassification that corrupted it. Addressing this demands reliability guarantees at the system level, not merely improved explanation at the interface.

The Error Cascading Problem. Consider a multi-agent workflow $W = \{A_1, A_2, \ldots, A_n\}$, where each agent A_i transforms an intermediate representation z_{i-1} into an output $z_i = A_i(z_{i-1}; \theta_i)$. A minor perturbation ϵ_i at an early stage does not merely persist—it acts as a multiplicative factor in downstream deviations [9]. This compounding is bounded by $\mathbb{E}[\|z_n - z_n^*\|] \leq \prod_{i=1}^{n}(1 + \epsilon_i)$, where z_n^* denotes the ground-truth trajectory. We term this *epistemic error cascading*: stochastic noise accumulates exponentially with pipeline depth, producing outputs that are fluent yet substantively incorrect [4]. From the human operator's perspective, the error is latent—detectable only through domain expertise the AI was meant to supplement.

Limitations of Existing Approaches. Current mitigation strategies present a dilemma for real-world deployment. Synchronous self-correction methods such as SelfCheckGPT and Self-Refine [17, 16] improve output quality via iterative critique-and-rewrite loops but introduce linear latency unsuitable for time-sensitive settings where sub-second responses are expected. Conversely, runtime guardrails like LLMGuard [10] and circuit-breaker patterns [25] act as binary filters: they block pathological outputs without recovering, leaving the human agent with nothing—arguably worse than a flawed output. Neither paradigm reconciles low latency with intelligent, autonomous recovery. This tension is not merely an engineering trade-off; it directly determines whether human operators can trust the system enough to use it effectively.

COCO: System-Level Reliability for Human-AI Teaming. We propose **COCO** (**C**ognitive **O**perating System with **C**ontinuous **O**versight), a framework that addresses reliability at the system level, operating transparently beneath the user interface. COCO departs from synchronous validation by adopting a decoupled asynchronous architecture that isolates the Monitoring Engine from

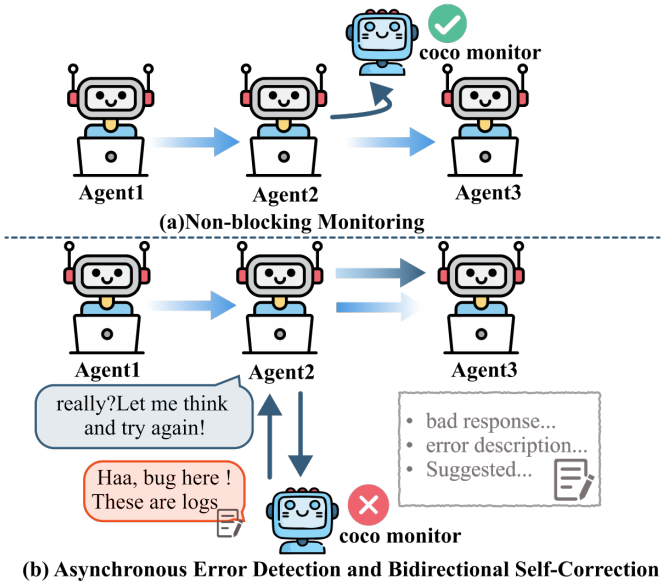


Fig. 1. Asynchronous monitoring intercepts intermediate execution errors, initiating a self-repair loop that prevents downstream error propagation from reaching the human operator.

the critical execution path, ensuring $O(1)$ overhead. This design is motivated by an HCI principle: for human operators to appropriately calibrate trust, the system must be reliably correct by default—not just occasionally correct with warnings that demand constant vigilance.

COCO orchestrates three synergistic mechanisms that target the distinct failure modes undermining human-AI collaboration: (1) The **Contextual Rollback Mechanism (CRM)** addresses stochastic errors by preserving execution snapshots (Ψ_i) and injecting diagnostic patches (Δ_i) for informed re-computation, halting epistemic cascading before it produces plausible-but-incorrect outputs. (2) The **Bidirectional Reflection Protocol (BRP)** prevents correction oscillation through a bounded single-turn consensus mechanism between execution and monitoring nodes, ensuring convergence without latency spikes that disrupt the operator's workflow. (3) **Heterogeneous Cross-Validation (HCV)** detects systematic biases through cross-architecture ensemble disagreement, compensating for blind spots that a single model family cannot self-detect.

The main contributions are summarized as follows:

– **System-Level Reliability Architecture:** We identify multi-agent error cascading as a structural barrier in human-facing AI pipelines and design COCO as a system-level reliability layer that operates transparently beneath the user interface, enabling informed error interception without disrupting operator workflows.

- **Tripartite Error-Correction Stack:** We introduce CRM for history-aware state restoration (addressing stochastic errors), BRP for stable actor-critic consensus with bounded overhead (preventing oscillatory behavior), and HCV for bias-resistant cross-model validation (detecting systematic failures).
- **Observational Integration Validation:** COCO has been tested in customer service and postal service workflow simulations, yielding qualitative insights into failure patterns and correction behavior beyond standard benchmarks.
- **State-of-the-Art Efficiency:** Extensive experiments show COCO achieves 95.1% of Titan-scale model performance with 30× parameter reduction and a 5.7% average improvement over strong baselines, while maintaining latency parity with unmonitored workflows.

2 Related Work

Multi-Agent Workflows. LLMs have shifted from isolated generation to compositional intelligence. Early methods like Chain-of-Thought and ReAct enabled stepwise reasoning [19, 22]. Recent frameworks scale this into Multi-Agent Systems (MAS), where heterogeneous models are auto-orchestrated into graph-based workflows [24]. Architectures such as AutoGen [20] and AgentVerse [7] distribute subtasks across specialized roles. However, this transition to sequential, interdependent collaboration increases execution graph depth, creating the structural conditions for epistemic error cascading.

Dynamics of Error Propagation. Recent studies validate the "snowballing" phenomenon: when a high-error-rate agent appears in a graph-structured workflow, global accuracy degrades exponentially, with declines exceeding 30% in linear structures [13]. Information-theoretic analysis reveals early-stage hallucinations as multiplicative noise [9, 8]. Counterfactual analysis suggests that precisely repairing key nodes can halve failure rates [5], motivating our Contextual Rollback Mechanism.

Auditing and Guardrail Paradigms. Current mitigation approaches fall into two categories. (1) *Synchronous Self-Correction* methods like SelfCheckGPT and Self-Refine [17, 16] use iterative generate–critique–rewrite loops, improving quality by 20% but introducing prohibitive linear latency. (2) *Runtime Guardrails* such as crowdsourcing-style voting [12] and LLMGuard [10] intercept pathological outputs but function as terminal blockers lacking the semantic context for intelligent recovery—the gap COCO fills.

HCI Perspectives on AI Reliability. User trust in automated systems depends critically on reliability and transparency [15]. When AI errs, users exhibit both overreliance and distrust, degrading decision quality [3]. Explanation quality and accuracy shape appropriate reliance, and Amershi et al. [1] codified 18 design guidelines for human-AI interaction. However, this research focuses on single-model interactions; the unique challenges of multi-agent cascading errors—where failures propagate silently across components—remain unexplored from an HCI perspective. COCO addresses this gap by providing system-level reliability guarantees that underpin effective human-AI teaming.

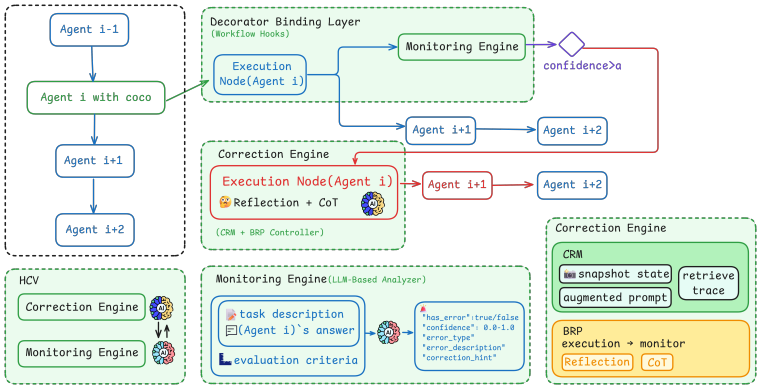


Fig. 2. Architectural overview of the COCO framework. The system is composed of three decoupled layers: (1) The **Decorator Binding Layer** facilitates non-intrusive, asynchronous oversight via agent-level hooks; (2) The **Monitoring Engine** leverages LLMs to synthesize structured diagnostic reports from execution outputs; and (3) The **Correction Engine** orchestrates state restoration, integrating CRM for contextual rollback with BRP and HCV to guarantee correction stability.

3 Methodology

Multi-agent workflows are inherently vulnerable to error cascading, yet current mitigation faces a dichotomy: synchronous validation imposes $O(n)$ latency, while stateless retries lack semantic context. COCO resolves this via native graph-based checkpointing with asynchronous decoupled monitoring, integrating seamlessly with existing frameworks to enforce quality guarantees with $\mathcal{O}(1)$ overhead.

3.1 Error Correction Components

COCO leverages the complementary strengths of its three core mechanisms to address these challenges: Contextual Rollback Mechanism (CRM), Bidirectional Reflection Protocol (BRP), and Heterogeneous Cross-Validation (HCV).

Contextual Rollback Mechanism To mitigate the stochastic degeneracy inherent in stateless retry heuristics, CRM implements a history-aware state recovery protocol. Unlike naive approaches that discard the execution context upon failure, CRM persists a structured checkpoint $\Psi_i = \langle s_i, P_i \rangle$ for each agent A_i, comprising the pre-execution state s_i and the original instruction set P_i.

Trigger and Formulation. Upon the detection of a high-confidence failure—formalized as the condition $\mathbb{I}(h_i = 1) \wedge (c_i \geq \alpha)$—CRM interrupts the downstream propagation and reverts the workflow to state s_i. Crucially, it constructs a Correction Patch Δ_i to condition the re-generation:

$$\Delta_i = \langle \delta_i, \gamma_i, z_i^{\text{fail}} \rangle \tag{1}$$

where δ_i represents the diagnostic rationale, γ_i denotes the corrective constraints, and z_i^{fail} serves as a negative sample.

Informed Re-computation. The re-execution mechanism is modeled as a counterfactual generation process:

$$z_i' = A_i(s_i, P_i^*; \theta_i) \quad \text{s.t.} \quad P_i^* = \mathcal{F}_{\text{aug}}(P_i, \Delta_i) \tag{2}$$

Here, the augmentation function $\mathcal{F}_{\text{aug}}$ integrates the patch Δ_i into the prompt manifold. By explicitly incorporating the failure mode z_i^{fail} and the diagnostic δ_i, CRM effectively modifies the optimization landscape, penalizing the previously taken path. This transforms the recovery process from a random resampling event into a guided local search, ensuring that z_i' diverges from locally convergent failure modes and effectively curbing epistemic error cascading.

Bidirectional Reflection Protocol To address the limitations of unidirectional validation, BRP implements a Dual-Phase Rectification Topology. Unlike iterative feedback that introduces unbounded latency, BRP operates as a strictly bounded, single-turn consensus mechanism, relying on reasoning depth (CoT) rather than iteration frequency to ensure quality:

Phase 1: Diagnostic Projection (Monitor Side). Upon detecting an anomaly, the monitoring engine $\mathcal{M}$ does not merely reject the output but projects a structured diagnostic vector $\mathbf{R}_i$ into the agent's context window. Formally, given the initial faulty output z_i^{fail} and agent state s_i, the monitor computes:

$$\mathbf{R}_i = \mathcal{M}(z_i^{\text{fail}}, \Psi_i) = \langle h_i, c_i, \tau_i, \delta_i, \gamma_i \rangle \tag{3}$$

where Ψ_i denotes structured execution snapshots, h_i indicates the presence of errors, c_i denotes the confidence score, τ_i identifies the error category, δ_i provides the diagnostic rationale (the "Why"), and γ_i offers directive constraints (the "How") to guide the correction.

Phase 2: Reflective Reconstruction (Agent Side). Instead of simply retrying, the execution agent A_i enters a Reflective Chain-of-Thought (CoT) mode. It consumes $\mathbf{R}_i$ to strictly condition its re-generation process. The corrected output z_i' is generated as:

$$z_i' = A_i(s_i, z_i^{\text{fail}}, \mathbf{R}_i; \theta_{\text{cot}}) \tag{4}$$

In this phase, the agent explicitly articulates an internal reasoning trace that aligns its intent with the constraints provided in γ_i before producing the final result.

Stability and Overhead. BRP enforces a strict DAG topology in correction logic, structurally severing the feedback edge after reconstruction. This eliminates control-loop oscillation while guaranteeing bounded $O(1)$ inference cost, avoiding latency spikes from indeterminate refinement.

Algorithm 1 Asynchronous Oversight and Contextual Recovery via COCO

Input: Workflow $\mathcal{G}(\mathcal{V}, \mathcal{E})$, Input query x, Configuration $\mathcal{D}$
Output: Final verified output z_n

Phase 1: Configuration Overlay Initialization
if $\mathcal{D}$ is NULL **then**
  $\mathcal{D} \leftarrow \text{SampleAndTest}(\mathcal{G}, \text{tasks})$ {Identify high-risk nodes by testing}
end if
$\tilde{\mathcal{V}} \leftarrow \text{LoadMonitoringNodes}(\mathcal{D})$ {Load Monitor Config}
Phase 2: Execution & Asynchronous Monitoring
for each agent node $A_i \in \text{Path}(\mathcal{G})$ **do**
  $z_i \leftarrow A_i(s_i, P_i; \theta_{exe})$ {Execute original agent task}
  if $A_i \in \tilde{\mathcal{V}}$ **then**
    Async Trigger: {Spawn parallel monitor thread}
    $\mathbf{R}_i = \langle h_i, c_i, \tau_i, \delta_i, \gamma_i \rangle \leftarrow \mathcal{M}(z_i, \Psi_i)$ {Generate diagnostics}
    if h_i is True **AND** $c_i > \alpha$ **then**
      Phase 3: Context-Aware Recovery (CRM + BRP)
      $\Delta_i \leftarrow \text{GenPatch}(\Psi_i, \mathbf{R}_i)$ {Patch Generation via context and diagnostics}
      $P_i^* \leftarrow \mathcal{F}_{aug}(P_i, \Delta_i)$ {Prompt augmentation via the patch}
      $z_i' \leftarrow A_i(s_i, P_i^*)$ {Reflective correction via BRP and CoT}
      Update $z_i \leftarrow z_i'$ and re-route downstream execution
    end if
  end if
end for
return z_n

Heterogeneous Cross-Validation In multi-agent collaboration, a single model often struggles to identify flaws in its own reasoning, leading to cognitive biases and blind spots that homogenous self-checks cannot detect. To address this, COCO introduces HCV, which establishes a cross-architecture feedback pathway by employing architecturally distinct models for execution and monitoring. When the monitor—with its fundamentally different inductive biases—disagrees with the executor's output, HCV interprets this divergence as a signal of potential systematic error. By leveraging complementary model architectures, HCV compensates for detection blind spots inherent to any single model family, improving overall error correction robustness through ensemble disagreement analysis.

3.2 Overall Architecture

COCO implements a three-layer decoupled architecture:

Monitor Binding Layer: Defines $\tilde{\mathcal{V}} \subseteq \mathcal{V}$ via non-invasive decorators, enabling asynchronous monitor triggering with $O(1)$ overhead and state isolation for multi-instance coordination.

Monitoring Engine: An LLM-based analyzer $\mathcal{M}(z_i, \Psi_i; \theta_{\text{mon}})$ performing multi-dimensional assessment (logical consistency, format compliance, content completeness), outputting the structured diagnostic $\mathbf{R}_i = \langle h_i, c_i, \tau_i, \delta_i, \gamma_i \rangle$.

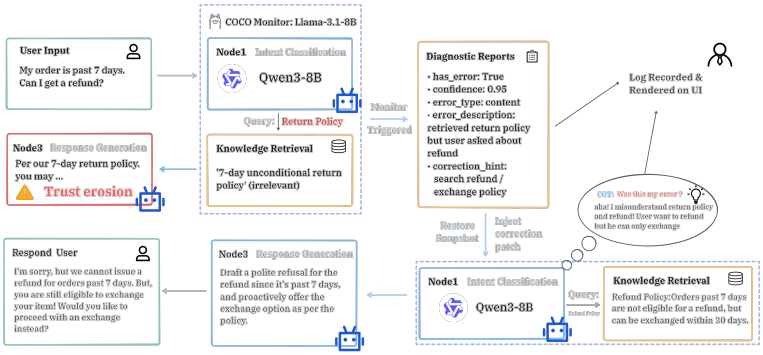


Fig. 3. Case study of a COCO-integrated customer service workflow. A multi-agent pipeline processes customer inquiries through intent classification (node1), knowledge retrieval (node2), and response generation (node3). COCO asynchronously monitors the retrieval agent: in the upper path, when COCO detects a missing-information error via bidirectional reflection between the execution and monitoring nodes, it alerts the operator, rolls back node2 with corrective context, and produces an accurate response—reducing the operator's cognitive burden of manually verifying intermediate reasoning. In the lower path without COCO's oversight, the error propagates downstream, yielding a fluent but substantively incorrect response that demands domain expertise to detect.

Correction Engine: Orchestrates CRM and BRP to manage state transitions, preserving snapshots Ψ_i and injecting diagnostics δ_i to generate augmented prompts $P_i^* = \mathcal{F}_{\text{aug}}(P_i, \Delta_i)$ for reflective re-computation z_i'.

COCO covers four error types (logic, format, content, systematic) and achieves seamless integration via annotation-based monitoring with minimal code changes. Figure 3 illustrates an end-to-end case study of COCO detecting and correcting a missing-information error in a customer service pipeline.

4 Experiments and Discussion

4.1 Datasets and Metrics

We benchmark COCO on diverse tasks covering both closed- and open-domain settings:

Closed-domain Tasks: GSM-Hard [16] features complex multi-step arithmetic problems requiring precise mathematical reasoning. **MMLU-Pro** [18] spans diverse knowledge domains through multiple-choice questions, testing factual recall and logical deduction. Both use accuracy as the evaluation metric.

Open-domain Tasks: MBPP [6] tasks agents with generating functionally correct code, demanding syntax, logic, and semantic validation. **CommonGen-Hard** [16] requires generating coherent sentences from disparate concepts, testing commonsense reasoning and creative expression. Evaluation combines executability, test pass rates, grammar, and semantic coherence.

Task Metric	GSM-Hard	MMLU-Pro	MBPP	CommonGen-Hard	AVG.	Improv. Overhead
Reference Models						
Qwen3-235B-A22B	**0.7210**	**0.7247**	**0.8120**	**0.8712**	**0.7822**	–
Qwen3-8B	0.6808	0.5885	0.6600	0.8115	0.6852	–
Llama-3.1-8B	0.3669	0.4803	0.6020	0.7700	0.5548	–
Multi-Agent Framework Baseline						
Aflow-Qwen3-8B	0.6399	0.6656	0.6500	0.8389	0.6986	–
Aflow-Llama3.1-8B	0.5648	0.4514	0.5580	0.7492	0.5809	–
COCO Framework (Proposed), notation *Exec / Mon*						
Qwen3-8B / Llama-3.1-8B	<u>0.6989</u>	<u>0.6869</u>	0.7440	0.8450	<u>0.7437</u>	+6.50%
Qwen3-8B / Qwen3-8B	0.6875	0.6660	<u>0.7660</u>	<u>0.8477</u>	0.7418	+6.20%
Llama-3.1-8B / Qwen3-8B	0.6023	0.5344	0.6040	0.8028	0.6359	+9.50%
Llama-3.1-8B / Llama-3.1-8B	0.5201	0.4562	0.5540	0.8081	0.5846	+0.63%

Table 1. Performance benchmarking across diverse datasets. We compare COCO against standard reference models and the state-of-the-art AFlow multi-agent baseline. For COCO rows, the *Exec / Mon* notation denotes the execution model and the monitoring model, respectively. All improvement metrics report gains relative to the corresponding AFlow workflow.

System Configuration	GSM-Hard	Avg. Latency (s)
AFlow-Qwen3-8B	0.6399	15.6
Qwen3-8B / Qwen3-8B	0.6875	22.7
Qwen3-8B / Llama-3.1-8B	**0.6989**	**15.0**

Table 2. Average latency comparison with Qwen3-8B backbone. Notation follows Table 1: *Exec / Mon*. COCO's heterogeneous configuration maintains parity with AFlow's efficiency while significantly enhancing task quality.

4.2 Implementation Details

AFlow Optimization. We employed the AFlow framework to automatically construct and optimize multi-agent workflows through Monte Carlo Tree Search-based exploration, generating DAG structures with performance-guided agent dependencies. Baseline configurations were augmented with analysis nodes for fair comparative evaluation.

COCO Decorator Architecture. Monitoring is implemented via a non-intrusive decorator pattern. A hybrid configuration strategy identifies high-risk nodes either through manual selection or an automated SampleAndTest protocol, generating an optimized overlay $\mathcal{D}$ that populates the monitoring set $\tilde{\mathcal{V}}$.

Heterogeneous vLLM Parallel Processing [14]. Local deployment uses vLLM's asynchronous framework for concurrent multi-agent processing: small-scale models (Qwen3-8B [21], Llama-3.1-8B [11]) run locally, while large-scale models (Qwen3-235B-A22B [21]) are accessed via API. All LLM inference uses temperature $T = 0.0$ to ensure deterministic, reproducible outputs and minimize stochastic variance across experimental runs.

Configuration	GSM-Hard	MBPP	Average
w/o Reflection w/o CoT	0.6880	0.6920	0.6900
w/ Reflection w/o CoT	0.6860	0.6800	0.6830
w/o Reflection w/ CoT	**0.7000**	0.7300	0.7150
w/ Reflection w/ CoT	0.6989	**0.7440**	**0.7215**

Table 3. Ablation study examining individual and combined contributions of COCO's abilities. Results demonstrate synergistic effects between Bidirectional Reflection Protocol and CoT reasoning.

4.3 Experimental Results

Table 1 demonstrates COCO's effectiveness: (i) consistent gains of 0.63%–9.5% over AFlow baselines; (ii) 95.1% of Qwen3-235B-A22B performance with only 8B parameters (30× reduction); (iii) bridging 53.9% of the gap between AFlow baselines and the large-model upper bound.

Computational Efficiency. Table 2 shows the heterogeneous **Qwen3-8B / Llama-3.1-8B** configuration achieves **15.0 s** average latency, matching the baseline **AFlow-Qwen3-8B** (15.6 s), confirming minimal overhead. The homogeneous **Qwen3-8B / Qwen3-8B** configuration incurs 22.7 s due to redundant inference from identical model paths. This highlights the efficiency advantage of heterogeneous deployment.

4.4 Ablation Study Results

Table 3 presents component-level analysis of COCO's innovations. The full configuration (w/ Reflection w/ CoT) achieves optimal 0.7215 average (+4.6% over baseline), validating the multiplicative benefit of CRM with complete BRP. CoT without BRP (0.7150) substantially outperforms BRP without CoT (0.6830) by 4.7%, confirming that enhanced reasoning provides the primary gain while BRP adds incremental refinement (+0.9%).

4.5 Discussion

Reflection-only Performance Analysis. The w/ Reflection w/o CoT configuration underperforms the baseline (0.6830, −1.0%), as incorporating erroneous outputs into the extended context window without CoT reasoning causes confusion and hallucination. This validates the necessity of the complete BRP design, where CoT provides the essential reasoning structure for effectively utilizing contextual rollback information.

Observational Deployment Testing. Beyond benchmarks, we deployed COCO in two simulated real-world scenarios: a customer service pipeline (intent classification → retrieval → response) and a postal logistics pipeline (address normalization → routing → notification). In both settings, COCO's asynchronous oversight improved response accuracy by intercepting upstream errors before

they cascaded into downstream agents, reducing the frequency of fluent-but-incorrect outputs that would otherwise reach human operators. The correction mechanisms operated transparently beneath the interface, eliminating the need for operators to manually verify intermediate reasoning steps and thereby reducing cognitive burden. These qualitative observations align with HCI findings that system-level reliability—rather than surface-level explanations—is key to fostering appropriate trust calibration in human-AI teaming [15, 3].

5 Limitations and Future Work

COCO exhibits several critical failure modes: (1) *Cascade failures* when multiple agents fail simultaneously; (2) *Detection blind spots* where subtle errors evade all three monitoring mechanisms; and (3) *Resource exhaustion* when error rates cause queue overflow. The framework assumes independent errors across agents, yet practical workflows exhibit correlated failures, violating convergence guarantees. From an HCI standpoint, the absence of direct human-subject evaluation means claims regarding trust calibration and cognitive burden await empirical validation; COCO's asynchronous transparency may also reduce operator situation awareness. Future work should develop meta-monitoring capabilities and controlled user studies measuring trust and decision quality. Ethically, COCO's layered logic reduces interpretability and may amplify biases when error patterns correlate with protected attributes.

6 Conclusion

COCO introduces a system-level framework for error detection and correction in multi-agent AI systems, operating transparently beneath the user interface. By integrating CRM, BRP, and HCV within an asynchronous decoupled architecture, COCO addresses error propagation with $O(1)$ overhead. Observational tests in customer service and postal service workflow simulations confirm its practical utility. Benchmark experiments demonstrate 5.7% average improvement and 95.1% of large-model performance with 30× parameter reduction. Ablation studies confirm the synergy between reflection-based monitoring and CoT reasoning. By providing reliability guarantees at the system level, COCO establishes a scalable technical foundation for multi-agent pipelines in human-facing applications, with future empirical validation of operator trust and decision quality remaining as essential next steps.

Acknowledgments. This research was supported by the 2024 Scientific Research Fund of Beijing University of Posts and Telecommunications Grant Number 510224070 and the 2024 Undergraduate Education and Teaching Reform Project of Beijing University of Posts and Telecommunications Grant Number 2024YB26.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Amershi, S., Weld, D., Vorvoreanu, M., et al.: Guidelines for human-AI interaction. In: CHI. pp. 1–13. ACM (2019)
2. Bansal, G., Wu, T., Zhou, J., et al.: Does the whole exceed its parts? The effect of AI explanations on complementary team performance. In: CHI. pp. 1–16. ACM
3. Buçinca, Z., Malaya, M.B., Gajos, K.Z.: To trust or to think: Cognitive forcing functions can reduce overreliance on AI in AI-assisted decision-making. In: CHI. pp. 1–14. ACM (2021)
4. Cemri, M., Pan, M.Z., et al.: Why do multi-agent LLM systems fail? In: NeurIPS Datasets and Benchmarks Track (2025)
5. Chen, J., Wang, Y., Wang, J., et al.: Understanding individual agent importance in multi-agent system via counterfactual reasoning. Proc. AAAI Conf. Artif. Intell. **39**(15), 15785–15794 (2025)
6. Chen, M., Tworek, J., et al.: Evaluating large language models trained on code
7. Chen, W., Su, Y., Zuo, J., et al.: AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors (2023)
8. Duan, J., Diffenderfer, J., et al.: UProp: Investigating the uncertainty propagation of LLMs in multi-step decision-making (2026)
9. Gan, Z., Liao, Y., Liu, Y.: Rethinking external slow-thinking: From snowball errors to probability of correct reasoning. In: ICML (2025)
10. Goyal, S., Hira, M., Mishra, S., et al.: LLMGuard: Guarding against unsafe LLM behavior. CoRR **abs/2403.00826** (2024)
11. Grattafiori, A., Dubey, A., Jauhri, A., et al.: The Llama 3 herd of models (2024)
12. Grunde-McLaughlin, M., Lam, M.S., Krishna, R., et al.: Designing LLM chains by adapting techniques from crowdsourcing workflows. ACM Trans. Comput. Hum. Interact. **32**(3), 27:1–27:57 (2025)
13. tse Huang, J., Zhou, J., et al.: On the resilience of LLM-based multi-agent collaboration with faulty agents (2025)
14. Kwon, W., Li, Z., Zhuang, S., et al.: Efficient memory management for large language model serving with PagedAttention. In: SOSP (2023)
15. Lee, J.D., See, K.A.: Trust in automation: Designing for appropriate reliance. Hum. Factors **46**(1), 50–80 (2004)
16. Madaan, A., Tandon, N., Gupta, P., et al.: Self-Refine: Iterative refinement with self-feedback. In: NeurIPS (2023)
17. Manakul, P., Liusie, A., Gales, M.: SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In: EMNLP (2023)
18. Wang, Y., Ma, X., et al.: MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In: NeurIPS Datasets and Benchmarks Track
19. Wei, J., Wang, X., Schuurmans, D., et al.: Chain-of-thought prompting elicits reasoning in large language models (2023)
20. Wu, Q., Bansal, G., Zhang, J., et al.: AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In: COLM (2024)
21. Yang, A., Li, A., Yang, B., et al.: Qwen3 technical report (2025)
22. Yao, S., Zhao, J., Yu, D., et al.: ReAct: Synergizing reasoning and acting in language models. In: ICLR (2023)
23. Yin, M., Wortman Vaughan, J., Wallach, H.: Understanding the effect of accuracy on trust in machine learning models. In: CHI. pp. 1–12. ACM (2019)
24. Zhang, J., Xiang, J., Yu, Z., et al.: AFlow: Automating agentic workflow generation. In: ICLR (2025)
25. Zou, A., Phan, L., et al.: Improving alignment and robustness with circuit breakers